GDNet V5: Byte-Level Traffic Classification for Security Product Discovery with Open-Set Rejection

lupingQAQ
Independent researcher
https://github.com/lupingQAQ
https://huggingface.co/luping123

Abstract

Enterprise networks rarely have a complete list of the security agents and appliances running on them. GDNet V5 is a small classifier that tries to recover part of that list from traffic. It reads the first five packets of a unidirectional flow, masks the IP addresses, and predicts one of 18 security products, a type, a tier, and a malicious-family label. A prototype distance decides whether the product prediction should be trusted. The released model is a four-arm ensemble; each arm has 2.86M parameters. On a 503-flow test split, product accuracy is 0.8787 and tier accuracy is 0.9245. The gate reaches AUROC 0.9784 and rejects all 18 unknown-product bins. The paper also reports an audit of six public benchmarks: five have roughly 80% exact train/test overlap, Tor has none. I do not claim state-of-the-art accuracy; the audit explains why absolute numbers on those five benchmarks are hard to interpret. Code and weights are Apache-2.0. Inference needs a CUDA GPU.

Keywords: encrypted traffic classification; security product discovery; open-set recognition; Gated DeltaNet; benchmark leakage

1 Introduction

Asset inventories are lists maintained by people. Traffic is a measurement taken from the wire. The two disagree often enough that an incident responder cannot rely on the list alone. Operations installs agents, the network team deploys appliances, and the central inventory is updated by neither group consistently.

I built GDNet V5 to answer one question: which security product produced this flow? It reads the first five packets, masks the IP addresses, and predicts a product, a type, a tier, and a malware family. Each arm has 2.86M parameters and runs on an 8 GB GPU. A prototype distance decides whether the product label should be trusted. Flows far from every known class are reported as unknown. The probe is a single C file; the Python package handles offline evaluation and the HTTP service.

Two choices shaped the design. The first is bytes. A tokenizer would add a parser and a vocabulary; bytes need neither. The released C probe is 42 KB and writes 1,600 bytes per flow. The second is the split. The pretraining pool leaves out validation and holdout flows. The threshold and the ensemble are chosen on dev, not test.

I also audited the six public benchmarks used in the internal comparison. Five contain about 80% exact train/test overlap. Tor is the only clean split. That finding changes how the absolute numbers should be read, so the paper reports a deduplicated evaluation instead of a leaderboard claim.

2 Related Work

2.1 Byte-level traffic classification

ET-BERT tokenizes datagrams and pretrains a Transformer with masked language modeling. YaTC represents flows as images and pretrains a masked autoencoder. NetMamba applies a unidirectional Mamba backbone to

packet sequences. MambaNetBurst removes tokenization and pretrains directly on bytes; its 2.7M-parameter model is the closest architectural relative of this work.

The backbone here follows the byte-level design of MambaNetBurst but uses Gated DeltaNet blocks. The released implementation is a trimmed GDN-only encoder. No upstream source is redistributed.

2.2 Foundation models

netFound learns packet-level representations from a large token corpus and evaluates frozen features on downstream tasks. It reports strong frozen-encoder results and explicit corrections for dataset shortcuts. The model is roughly two orders of magnitude larger than one GDNet arm. An internal comparison under a fixed five-packet budget was competitive, but it is not reproduced here. Input budget and split construction differ enough that cross-paper numbers are hard to compare.

2.3 Open-set recognition

Prototypical networks classify a sample by its distance to class prototypes. I use three sub-prototypes per class to represent intra-class variation. At inference, the distance is one minus the maximum cosine similarity to any prototype. A threshold calibrated on dev decides whether to trust the closed-set product head. The method needs no unknown-class training data.

2.4 Dataset leakage

Trustee and Pcap-Encoder show that network ML systems often exploit dataset shortcuts, including packet-level splits and timestamp correlations. netFound reports corrected splits for several public datasets. The audit in Section 5 extends this line with exact byte-level duplicate counts for six widely used traffic benchmarks. It is independent of any model.

3 Method

3.1 Input

I keep the first five packets, not the whole flow. The probe emits a vector after packet five, and waiting for the connection to close would add latency to every verdict. Five packets also cover the handshake and the first application message, which is where most of the product signal sits. The vector is 1,600 bytes: 320 bytes per packet, 80 for the IPv4 and TCP/UDP headers, 240 for the payload. I zero bytes 12:20, the IPv4 addresses. Without the mask the model can learn host identity; with it, the input is portable across sites.

The parser is dpkt. It handles Ethernet, VLAN, and raw IPv4 in pcap or pcapng. I did not use libpcap because the Python wheel is easier to install and the parser is not on the hot path.

Property	Value
Flow length	1600 bytes
Packets per flow	5
Bytes per packet	320
Header bytes per packet	80
Payload bytes per packet	240
Address handling	IPv4 source and destination zeroed
Direction	Unidirectional

3.2 Backbone

One arm is four GDN blocks at d_model=256 and d_state=16, with a 256-dimensional byte embedding. The CLS token is appended after the last byte; the final CLS state is the feature. The state dict has 103 entries. Fifty-two belong to the GDN blocks; the rest are the embedding, the heads, and the training-only domain head. I load with strict=False, so the domain head can stay in the checkpoint without being instantiated at inference.

```
bytes (1600)
  -> ByteEmbed (256)
  -> [bytes, CLS] + positional embedding
  -> 4 x GDN (d_model=256, d_state=16)
  -> CLS feature (256)
```

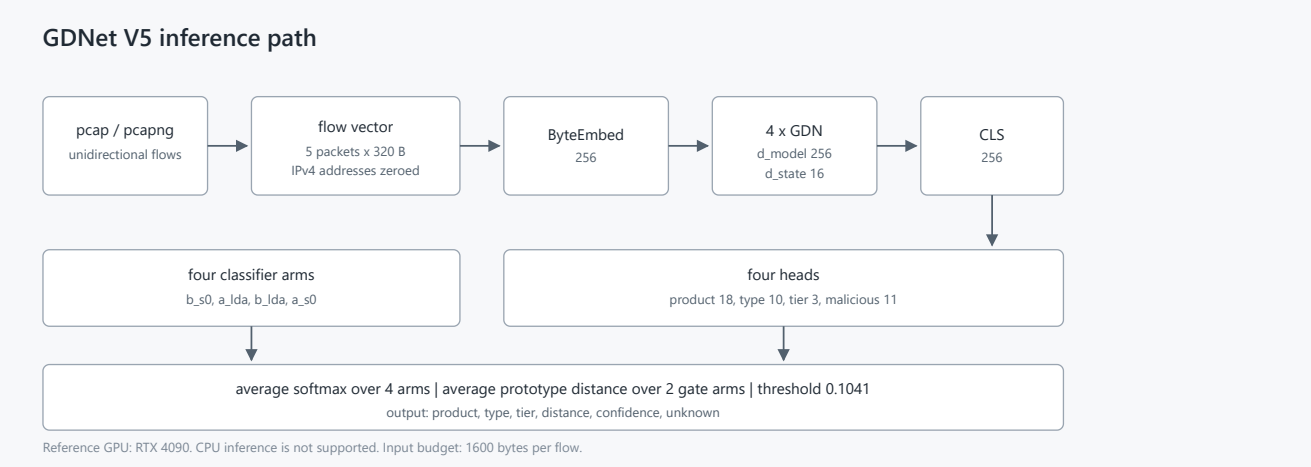


Figure 1. Inference path. The four-arm ensemble and two-arm gate are described in Section 3.5.

3.3 Task heads

The product head is a three-layer MLP: 256 inputs, 512 hidden units, 18 outputs. The tier head is a single linear layer from 256 to 3. The type and malicious heads use the same 512-unit hidden layer, with 10 and 11 outputs. The head weights are about 0.4M parameters, small compared with the 2.4M encoder. The tier head is cheap and gives a coarser label; the malicious head is not used in the main results.

Head	Outputs	Purpose
Product	18	Security product or service
Type	10	WAF, behavior management, bastion host, NDR, zero trust/VPN, EDR/HIDS, antivirus, browser, file transfer, remote administration
Tier	3	Security appliance, security software, productivity software
Malicious	11	Benign plus ten malware families

3.4 Open-set decision

The open set uses a 54 x 256 prototype matrix per gate arm. I build it with KMeans over the training embeddings, using three clusters per class. At inference the feature is normalized; the distance is one minus the maximum cosine similarity over the 54 rows. The matrix is about 54 KB per arm. The threshold is 0.1041, the 68th percentile of dev distances. It is not a hand-picked value; it comes from the dev quantile. A new site should recalibrate it.

```
d = 1 - max cosine(feature, prototype)
```

3.5 Ensemble

The ensemble uses four arms. I dropped the two hard-negative arms because their gate distance moved when I changed the threshold, even though their AUROC was higher. The four remaining arms differ in pretraining data and loss weights. At inference I average four softmax vectors and two distance vectors. The ensemble has 11.44M parameters and a 46 MB footprint.

3.6 Training

Pretraining used 33,184 training-only flows. The pool excludes validation and holdout flows; the build script checks the intersection and fails if it is non-zero. SFT ran for 60 epochs, RL for 300 episodes. I trained on four RTX 4090s.

The training code is not in v1.0.0. It assumes the lab data layout, and I would rather publish a tested inference path than an untested training path. The checkpoints, prototypes, threshold, and evaluation script are enough to reproduce inference.

4 Evaluation

4.1 Data and splits

The labeled set has 40,283 flows across 18 classes, all from lab captures. The validation set has 1,006 flows from 14 held-out sessions. I split it into dev and test at flow level, 503 each. Threshold, checkpoint, and ensemble choices use dev. Test is used once.

The 18 classes are `windows_defender`, `clamav_freshclam_win`, `clamav_freshclam`, `velociraptor`, `velociraptor_win`, `osquery`, `wazuh_agent`, `wireguard`, `openvpn`, `stunnel`, `squid_proxy`, `ntopng`, `guacamole_mgmt`, `waf_modsecurity`, `suricata_update`, `web_http`, `ssh`, and `ftp`. Captures came from lab hosts and virtual machines. Each product was exercised through its normal update or management path, not through synthetic payloads.

The split is at flow level, not session level. Each class has one held-out session. The numbers describe those sessions, not a new organization.

4.2 Metrics

- Product accuracy counts exact product labels.
- Tier accuracy counts exact tier labels.
- Macro-F1 averages over classes with at least one test flow.
- Gate AUROC compares known flows with unknown and novel flows.
- Unknown rejection is the share of unknown bins above the threshold.
- Malicious macro-F1 uses the deduplicated USTC-TFC2016 test split.

4.3 Results

Metric	Test (503 flows)	All validation (1006 flows)
Product accuracy	0.8787	0.8817
Tier accuracy	0.9245	0.9334
Macro-F1, supported classes	0.7621	0.8240
Gate AUROC	0.9784	0.9820 (dev)

Product accuracy is 0.8787 on test and 0.8817 on all validation. Tier accuracy is 0.9245 and 0.9334. The gate reaches AUROC 0.9784 on test and 0.9820 on dev. Macro-F1 is 0.7621 on test and 0.8240 on all validation. The test number is unstable: `wireguard` has one flow, so a single error moves the class score by a large amount. I would use the all-validation value as the summary.

All 18 unknown bins are rejected. The operating point stays stable from q=0.64 to q=0.75. The malicious head reaches 0.9702 macro-F1 on the deduplicated USTC test set (n=1242); the binary F1 is at least 0.994.

4.4 Acceptance and enrollment

The acceptance set has 264 product bins and 18 unknown bins. The raw ensemble gets 8 of 10 products right. A timing rule that uses inter-arrival times raises this to 9, but the rule is not in the release. I did not want to ship a second model without its own calibration. Unknown bins are rejected at 100% in both settings.

`stunnel` is the failure: 0 of 30 bins. `wireguard` has two bins; I treat that as noise.

The HTTP service returns 6/6 known flows on FLW1 records. Enrollment reaches 8/8 after three sub-prototypes for `smb`.

4.5 Throughput and memory

Configuration	Throughput	Latency at batch 1	Peak VRAM at batch 128
Single arm	1,017 flows/s	10.7 ms	3.37 GB
Four-arm ensemble	253 flows/s	43.1 ms	3.40 GB
Distilled student	1,011 flows/s	10.6 ms	3.37 GB

On one RTX 4090, a single arm runs at 1,017 flows/s and the four-arm ensemble at 253 flows/s. Batch-1 latency is 10.7 ms and 43.1 ms. The ensemble peaks at 3.4 GB at batch 128. The ensemble rate is below the probe's capture rate, so deployment needs sampling, multiple cards, or per-flow triggering.

5 Public Split Audit

5.1 Method

I hash every flow in the distributed train, validation, and test directories. Two hashes are used: MD5 of the PNG file bytes, and MD5 of the decoded grayscale pixels. The two hashes agree. Duplicates are exact byte-level matches, not near-duplicates. The input unit is 1600 bytes, matching the model input.

The audit covers six benchmarks used in the internal comparison: ISCXVPN2016, ISCXTor2016, USTC-TFC2016, CICIoT2022, CrossPlatform-Android, and CrossPlatform-iOS. The scripts are released with the code.

5.2 Results

Benchmark	Train	Test	Train ∩ test	Share of test	Clean test
ISCXVPN2016	13,281	1,384	1,106	79.9%	278
ISCXTor2016	11,655	1,458	0	0%	1,458
USTC-TFC2016	37,323	6,662	5,605	84.1%	1,057
CICIoT2022	18,734	1,951	1,571	80.5%	380
CrossPlatform-Android	44,700	4,656	3,733	80.2%	923
CrossPlatform-iOS	40,379	4,205	3,396	80.8%	809

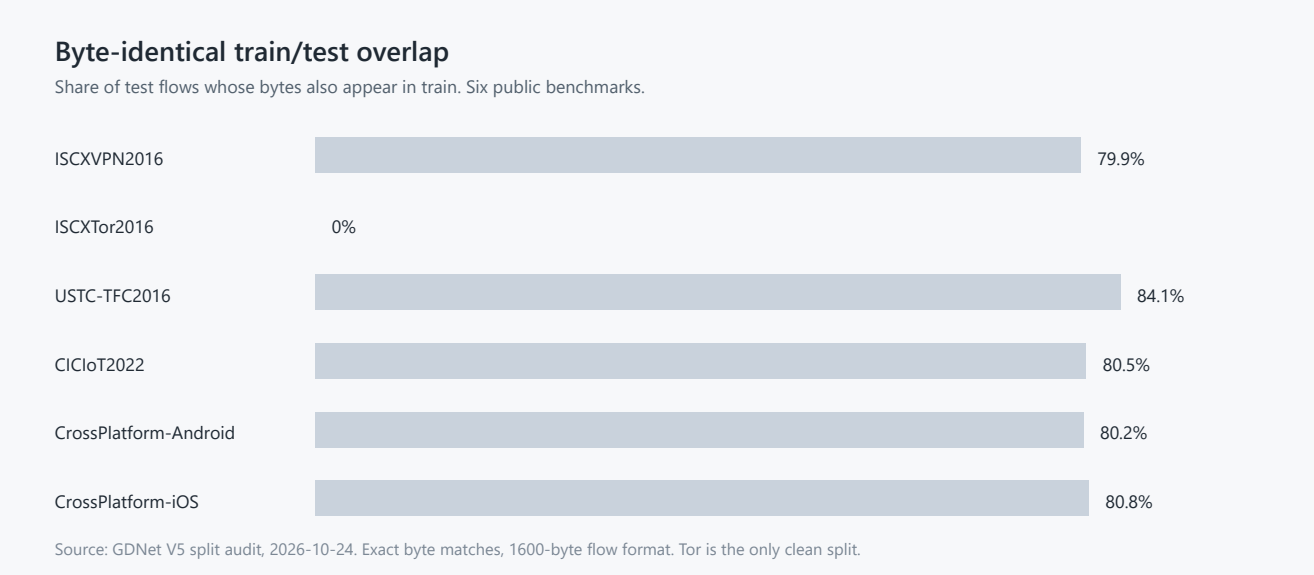


Figure 2. Exact byte-level train/test overlap in six public benchmarks. Tor is the only clean split.

Label conflicts across splits are zero. Every duplicate stays within its class. The likely cause is independent sampling from the same flow pool without cross-split deduplication. That is a property of the data distribution, not of a single paper.

5.3 Impact

Absolute accuracy on these benchmarks is inflated. About 80% of the test flows are memorization cases in five of the six benchmarks. Same-split comparisons remain valid as same-protocol comparisons. They are not evidence of generalization.

Tor is the only clean benchmark in the set. Comparisons on Tor are the strongest evidence in the internal report. For the other benchmarks, I report both the original split and a deduplicated test subset.

5.4 Deduplicated evaluation

I reevaluate three affected benchmarks on the deduplicated test subset. The clean test sizes are 278 flows for VPN, 1,057 for USTC, and 380 for CIoT. Tor is already clean and is not rerun.

Benchmark	Clean test	Accuracy	Macro-F1	Full-set macro-F1	Change
ISCXVPN2016	278	0.9376	0.9418	0.9861	-4.4 pp
USTC-TFC2016	1,057	0.9981	0.9671	0.9949	-2.8 pp
CICIoT2022	380	0.9868	0.9823	0.9965	-1.4 pp

The drop is consistent with removing memorized flows. The internal report also evaluates an external baseline on the same clean subsets and finds that the relative gap does not shrink. That comparison is not reproduced here because the external checkpoints and data are not redistributed.

5.5 What did not work

Attempt	Result	Decision
Auxiliary pretraining (field, order, reconstruction)	Probe +9.35 points; downstream E1 below E0+LDA	Not used
Timing rescue	Acceptance 8 to 9; unstable at nearby thresholds	Not released
Distillation	0.8231 validation accuracy; 5.6 points below the ensemble	Experimental only
`stunnel` threshold tuning	0/30 at every threshold	Needs data or enrollment

These failures shaped the release. A larger model or a more complex gate would not fix the split problem, and I did not add components that only helped the dev set.

6 Limitations

The biggest limitation is the split, not the model. Each class has one held-out session, so the test numbers describe those sessions, not a new site. A second session per class would be more useful than another point of accuracy.

`stunnel` is not fixed. I tried enrollment, and the 8/8 result on `smb` shows the mechanism works, but `stunnel` needs its own data.

The timing rule used in the 9/10 acceptance result is not in v1.0.0. The released ensemble reproduces the 8/10 raw result.

CPU inference is not supported. The GDN backbone uses Triton kernels and needs a CUDA GPU. A CPU port would be a separate implementation, not a build flag.

The model was trained on lab traffic. Performance on production networks with different software versions, proxies, or NAT configurations is unknown.

The label set is closed at 18 products plus an unknown class. Products outside the set are rejected or misclassified.

Training code is not in v1.0.0. The checkpoints, prototypes, threshold, and evaluation script are enough for inference and evaluation, not for retraining. The training scripts depend on the lab data layout and need cleanup before they are useful.

The public benchmark comparison is same-protocol. I do not claim state-of-the-art accuracy.

7 Release and Reproducibility

- Code: https://github.com/lupingQAQ/gdnet
- Weights and model card: https://huggingface.co/luping123/gdnet-v5
- Release: https://github.com/lupingQAQ/gdnet/releases/tag/v1.0.0
- License: Apache-2.0 for code and weights, CC-BY-4.0 for documentation.
- Checksums: `SHA256SUMS` in the model repository and the release assets.
- Input format: pcap/pcapng, first five packets, IPv4 addresses masked.
- Hardware: NVIDIA GPU with CUDA. Reference GPU is an RTX 4090; 4 GB is the minimum at batch 32.

The repository does not contain captures. The weights were trained on lab traffic and public benchmark training splits. No customer traffic is included.

7.1 Reproduction notes

Download the weights from Hugging Face and run the `gdnet` CLI on a capture. The released weight directory is 94.6 MB; the four-arm ensemble is 46 MB. The CLI prints one row per flow and can write the full result as JSON. The evaluation script expects a directory of 1600-byte flow bins and a CSV label file. The split itself is not redistributed. Checkpoint SHA256 values are in the model repository. The code was tested with Python 3.12 and PyTorch 2.14 on an RTX 4090. The CPU path is intentionally absent.

8 Ethics and Dual Use

The model identifies security products on a network. That is a defensive capability, but traffic classification can also reveal information about a network's defenses. Use the model only on networks you own or have written permission to monitor. Do not upload captures to public demos. Do not use the model for unauthorized surveillance.

The model outputs probabilities and distances, not facts. Low-confidence and unknown results should be reviewed by a human. The malicious head is a secondary output and is not a replacement for an IDS, EDR, or vulnerability scanner.

IPv4 addresses are masked before inference. The model does not inspect payload semantics beyond byte patterns. It does not identify users or content.

Appendix A: Label Set

Product	Type	Tier
windows_defender	Antivirus	Security software
clamav_freshclam_win	Antivirus	Security software
clamav_freshclam	Antivirus	Security software
velociraptor	EDR / HIDS	Security software
velociraptor_win	EDR / HIDS	Security software
osquery	EDR / HIDS	Security software
wazuh_agent	EDR / HIDS	Security software
wireguard	Zero trust / VPN	Security appliance
openvpn	Zero trust / VPN	Security appliance
stunnel	Zero trust / VPN	Security appliance
squid_proxy	Internet behavior management	Security appliance
ntopng	NDR	Security appliance
guacamole_mgmt	Bastion host	Security appliance
waf_modsecurity	WAF	Security appliance
suricata_update	NDR	Security appliance
web_http	Browser	Productivity software
ssh	Remote administration	Productivity software
ftp	File transfer	Productivity software

References

1. MambaNetBurst: Direct Byte-level Network Traffic Classification without Tokenization or Pretraining. arXiv:2605.11034, 2026.
2. NetMamba: Efficient Network Traffic Classification via Pre-training Unidirectional Mamba. arXiv:2405.11886, 2024.
3. ET-BERT: A Contextualized Datagram Representation with Pre-training Transformers for Encrypted Traffic Classification. arXiv:2110.02633, 2021.
4. YaTC: Yet Another Traffic Classifier. arXiv:2305.19077, 2023.
5. netFound: Foundation Model for Network Security. arXiv:2310.17025, 2023 (v5, 2026).
6. Gated Delta Networks: Improving Mamba2 with the Delta Rule. arXiv:2412.06464, 2024.
7. Prototypical Networks for Few-shot Learning. arXiv:1703.05175, 2017.
8. Pcap-Encoder: The Sweet Danger of Sugar. arXiv:2507.16438, 2025.
9. Trustee: AI/ML for Network Security: The Emperor Has No Clothes. CCS 2022.
10. NetBench: A Benchmark for Network Traffic Classification. FMSys 2024.
11. flash-linear-attention. https://github.com/fla-org/flash-linear-attention.
12. PyTorch. https://pytorch.org.